# Data Wrangling in R

Lucy Gagnon

Today's agenda: Manipulating data objects; using the built-in functions, doing numerical calculations, and basic plots; reinforcing core probabilistic ideas.

***General instructions for homeworks***: Please follow the uploading file instructions according to the syllabus. You will give the commands to answer each question in its own code block, which will also produce plots that will be automatically embedded in the output file. Each answer must be supported by written statements as well as any code used. Your code must be completely reproducible and must compile.

***Advice***: Start early on the assignment and it is recommended that you start early. Not late assignments will be accepted.

***Commenting code*** Code should be commented. See the Google style guide for questions regarding commenting or how to write code https://google.github.io/styleguide/Rguide.xml.

***R Markdown Test***

0. Open a new R Markdown file; set the output to HTML mode and "Knit". This should produce a web page with the knitting procedure executing your code blocks. You can edit this new file to produce your homework submission.

   **Done**. I set 'output: html_document' in the document header.

***Working with data***

Total points on assignment: 3 (reproducibility) + 22 (Q1) + 9 (Q2) + 3 (Q3) = 37 points

1. *(22 points total, equally weighted) The data set* ***rnf6080.dat*** *records hourly rainfall at a certain location in Canada, every day from 1960 to 1980.*

   1. *Load the data set into R and make it a data frame called* `rain_df`*. What command did you use?*

```
# import packages
library(readr)

# load dataset and make into dataframe
rain_df <- read.table('data/rnf6080.dat', header=FALSE)
head(rain_df) # print preview
```

```
##   V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18 V19 V20 V21
## 1 60  4  1  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
## 2 60  4  2  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
## 3 60  4  3  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
## 4 60  4  4  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
## 5 60  4  5  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
## 6 60  4  6  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
##   V22 V23 V24 V25 V26 V27
## 1   0   0   0   0   0   0
```

```
## 2   0   0   0   0   0   0
## 3   0   0   0   0   0   0
## 4   0   0   0   0   0   0
## 5   0   0   0   0   0   0
## 6   0   0   0   0   0   0
```

```
class(rain_df) # check dataframe format
```

```
## [1] "data.frame"
```

I loaded the data set using the base R 'read.table' command.

2. *How many rows and columns does* `rain_df` *have? How do you know? (If there are not 5070 rows and 27 columns, you did something wrong in the first part of the problem.)*

```
# check the size of rain_df
dim(rain_df)
```

```
## [1] 5070   27
```

The rainfall data frame has 5070 rows and 27 columns. I used the 'dim' command to check the dimensions of the data frame, which gives the number of rows and the first output and number of columns as the second.

3. *What command would you use to get the names of the columns of* `rain_df`*? What are those names?*

```
# extract column names of rain_df
colnames(rain_df)
```

```
##  [1] "V1"  "V2"  "V3"  "V4"  "V5"  "V6"  "V7"  "V8"  "V9"  "V10" "V11" "V12"
## [13] "V13" "V14" "V15" "V16" "V17" "V18" "V19" "V20" "V21" "V22" "V23" "V24"
## [25] "V25" "V26" "V27"
```

I used the 'colnames()' command to obtain the column names of rain_df. The column names are "V1", "V2",... "V27"

4. *What command would you use to get the value at row 2, column 4? What is the value?*

```
# extract value at row 2, column 4
value<-rain_df[2,4]
# print value
print(value)
```

```
## [1] 0
```

I used data frame indices to extract the value at row 2, column 4.The value is 0.

5. *What command would you use to display the whole second row? What is the content of that row?*

```
# extract whole second row
second_row_values<-rain_df[2,]
# print second row
print(second_row_values)
```

```
##   V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18 V19 V20 V21
## 2 60  4  2  0  0  0  0  0  0   0   0   0   0   0   0   0   0   0   0   0   0
##   V22 V23 V24 V25 V26 V27
## 2   0   0   0   0   0   0
```

To display the whole second row, I used the indexing command, with nothing in the column index to extract the entire row. The second row includes V1 = 60, V2 = 4, V3 = 2, and the rest of the columns are zeroes.

6. *What does the following command do?*

```
names(rain_df)<-c("year", "month", "day", seq(0,23))
# print result
print(names(rain_df))
```

```
##  [1] "year"  "month" "day"   "0"     "1"     "2"     "3"     "4"     "5"
## [10] "6"     "7"     "8"     "9"     "10"    "11"    "12"    "13"    "14"
## [19] "15"    "16"    "17"    "18"    "19"    "20"    "21"    "22"    "23"
```

The above command adds column names to rain_df. The rows then give rainfall for a certain hourly datetime by year, month, day, and hour (0,23).

7. *Create a new column called* `daily_rain_fall`*, which is the sum of the 24 hourly columns.*

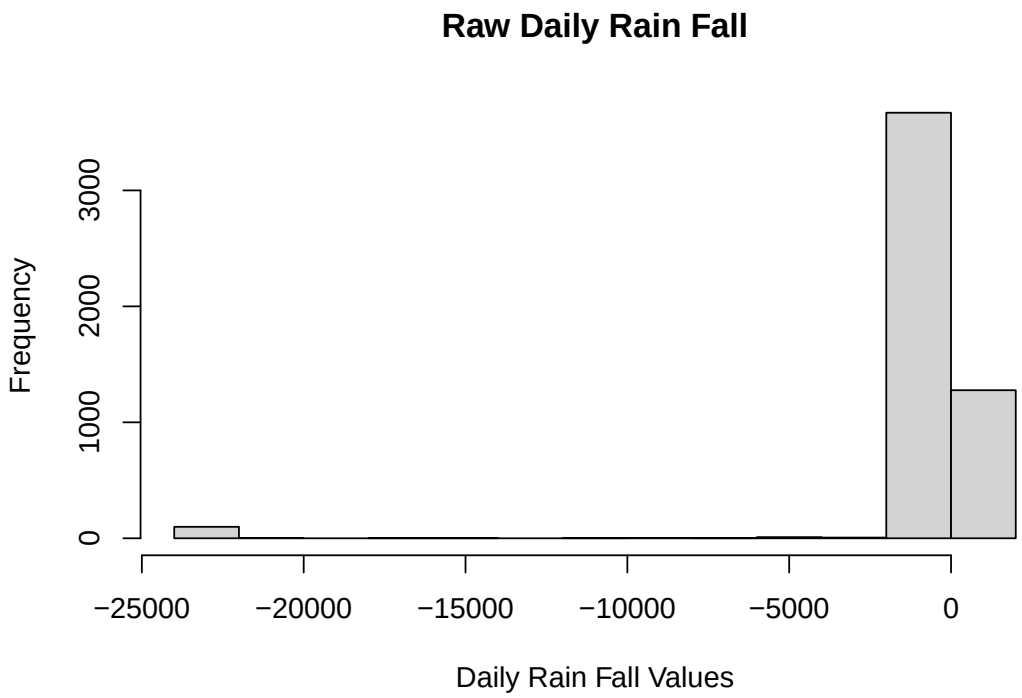
```
# create a new column called 'daily_rain_fall' and sum 24 hourly columns
# I used ChatGPT for help to make sure R didn;t interpret the 0-23 as indices
hour_cols <- as.character(0:23) # identify hour columns
# my code
rain_df$daily_rain_fall <- rowSums(rain_df[, hour_cols])
# check result
head(rain_df$daily_rain_fall)
```

```
## [1] 0 0 0 0 0 0
```

Here, we created a new column, called 'daily_rain_fall', where we summed the 24 hourly columns.

8. *Give the command you would use to create a histogram of the daily rainfall amounts. Please make sure to attach your figures in your .pdf report.*

```
# create a histogram of daily rainfall amounts
hist(rain_df$daily_rain_fall, xlab = "Daily Rain Fall Values",
     main = "Raw Daily Rain Fall")
```




I used the 'hist' command to create a histogram of the daily rainfall amounts.

9. *Explain why that histogram above cannot possibly be right.*

   The histogram cannot possibly be right because the plot shows negative daily rain fall, which is non-physical.

10. *Give the command you would use to fix the data frame.*

```
# import package
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

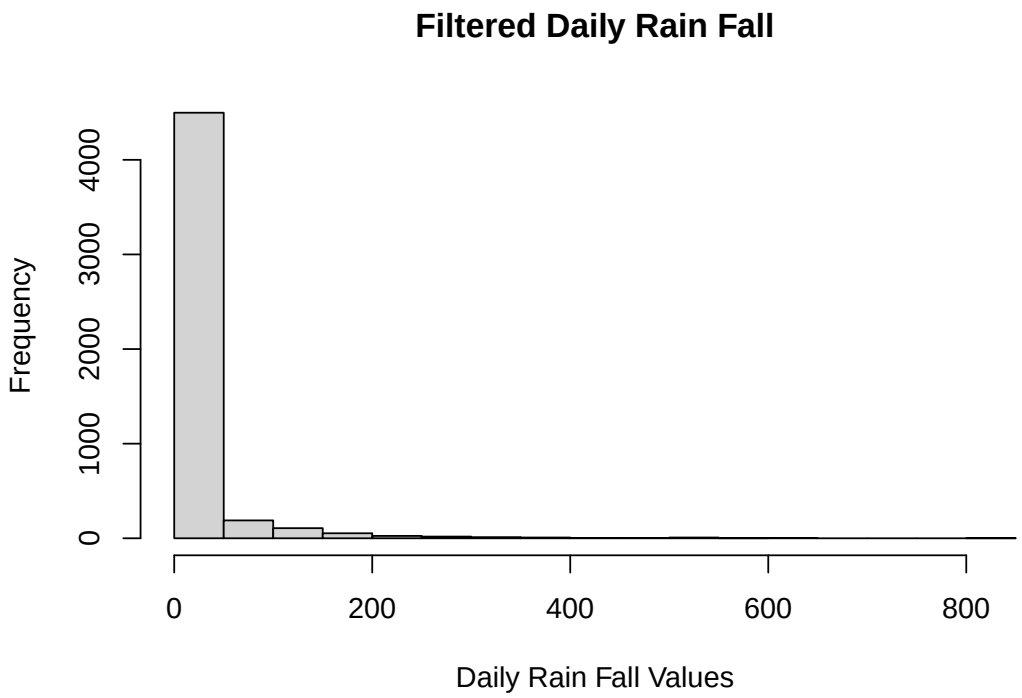
```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# filter for non-negative rainfall totals
filtered_rain_df <- rain_df %>% filter(daily_rain_fall >= 0)
```

11. *Create a corrected histogram and again include it as part of your submitted report. Explain why it is more reasonable than the previous histogram.*

```
# create the corrected histogram
hist(filtered_rain_df$daily_rain_fall, xlab = "Daily Rain Fall Values",
     main = "Filtered Daily Rain Fall")
```




This histogram is more reasonable because all daily rainfall values are physically plausible values (non-negative).

***Data types***

2. *(9 points, equally weighted) Make sure your answers to different parts of this problem are compatible with each other.*

    2. *For each of the following commands, either explain why they should be errors, or explain the non-erroneous result.*

```
x <- c("5","12","7")
max(x)
```

```
## [1] "7"
```

```
sort(x)
```

```
## [1] "12" "5"  "7"
```

```
#sum(x)
```

For the vector assignment, this is likely an error because you are supplying numbers as strings, when you would most likely prefer numeric datatypes.

Taking the maximum of this vector $x$ is an error because the maximum value in $x$ should be 12. However, because the numbers are supplied as strings, R picks the last input as the maximum of $x$, which is 7.

The *sort(x)* command is an error because the command sorts the strings by the first digit which appears, not numerically.

The s*um(x)* command produces an error since you cannot sum strings. I commented it out so the pdf document will render.

3. *For the next two commands, either explain their results, or why they should produce errors.*

```
y <- c("5",7,12)
#y[2] + y[3]
```

This vector $y$ includes mixed data types (a string and numeric data types). This will likely cause errors when you try to use "5" in numeric operations.

As predicted, the inclusion of the string in the vector reuslts in an error when trying to sum the numeric entries, so I commented the line out for rendering.

4. For the next two commands, either explain their results, or why they should produce errors.

```
z <- data.frame(z1="5",z2=7,z3=12)
z[1,2] + z[1,3]
```

```
## [1] 19
```

This works because dataframes can handle mixed data types. So, you are able to sum the two numeric entries.

3. *(3 pts, equally weighted).*

2. *What is the point of reproducible code?*

   The point of reproducible code is so others (and your future self) can understand what your code does and re-create it themselves (or yourself).

3. *Given an example of why making your code reproducible is important for you to know in this class and moving forward.*

   Making your code reproducible is valuable in this class and moving forward because you could reference your code at a later date, such as when reviewing for exams or working on a research project. Reproducible code allows you to can quickly and easily understand what you did in the past.

4. *On a scale of 1 (easy) – 10 (hard), how hard was this assignment. If this assignment was hard ($> 5$), please state in one sentence what you struggled with.*

   This assignment was a 3 in difficulty. I had to look some commands up, but they were quick to incorporate once I knew them.